Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2113 Diseño Detallado de Software

Entrega 1: Octopath Traveler

Javiera Ignacia Pinto Santa María
Matías Andrés Poblete Farías

Introducción

En esta entrega se deberá implementar el flujo principal del combate, lo que abarca el flujo completo del combate pero sin habilidades activas ni de apoyo, utilizando solo ataques básicos. En resumen, se debe implementar lo siguiente:

1. Flujo del combate, desde el inicio hasta que haya un vencedor.
2. Validación de equipos.
3. Acciones atacar y huir.
4. Cancelar acciones
5. Orden de turnos por *stat* Speed.
6. Leer las unidades desde archivos json.
7. Cálculos de daños sin debilidades, `Boost Points` ni `Breaking Point`.

Los tests correspondiente a esta entrega buscan simular combates en donde viajeros usan solo ataques básicos y bestias usan una misma acción simple. Por lo tanto, es válido trabajar bajo los siguientes supuestos en esta entrega:

- Los viajeros solo usarán ataques básicos.
- Los viajeros pueden tener habilidades activas y seleccionar la opción de usar habilidad, sin embargo en este último caso siempre se escogerá cancelar.
- Los viajeros no tendrán habilidades pasivas de ningún tipo.
- En ningún caso se utilizará **BP**, dado que las mecánicas del `boosting` no forman parte de esta entrega.
- Las bestias solo tienen la habilidad `Attack` que se comporta de forma similar a un ataque básico.
- En ningún caso se atacará la debilidad de una bestia, por tanto el cálculo de daño bajo debilidades y las mecánicas asociadas al `Breaking Point` no forman parte de esta entrega.

Test cases

Para esta entrega debes completar los siguientes grupos de tests:

- E1-BasicCombat
- E1-InvalidTeams
- E1-RandomBasicCombat

Formato Equipos

Lo primero que debe hacer tu programa es pedirle al usuario que seleccione un equipo. Cada grupo de test cases tiene un conjunto de equipos posibles diseñados para verificar el correcto funcionamiento de tu programa. Los equipos también se encuentran en `data.zip`. Por ejemplo, los equipos usados en los test cases `E1-BasicCombat` están en `data/E1-BasicCombat/`.

Para saber qué equipos se pueden utilizar, la clase `Game.cs` (que es la entrada a tu programa) recibe en su constructor el parámetro `teamsFolder`. Este parámetro contiene la ruta a la carpeta con todos los equipos disponibles según el test case que se esté ejecutando. Esta carpeta tendrá un archivo `.txt` por cada equipo posible para cada jugador. Los equipos deben ser mostrados al usuario y luego se le debe pedir como input que elija alguno de ellos. En el siguiente ejemplo, hay 6 equipos posibles y el usuario elige el equipo `0`:

```
Elige un archivo para cargar los equipos
0: 001.txt
1: 002.txt
2: 003.txt
3: 004.txt
4: 005.txt
5: 006.txt
INPUT: 0
```

Cada uno de estos archivos tendrá los datos de un equipo de personajes para el jugador y un equipo de enemigos a los cuales se enfrentará. El formato del equipo será un archivo `.txt`.

La primera línea será el texto `Player Team`, luego de ello cada línea será un personaje del equipo del jugador, que podrá tener hasta dos secciones para habilidades. La primera estará entre paréntesis `( )` la cual tendrá las habilidades activas que posee el viajero, separadas por comas. La segunda estará entre corchetes `[ ]` la cual tendrá las habilidades de apoyo de la unidad.

Si un viajero no posee algunas de las secciones descritas, entonces no posee ninguna habilidad de ese tipo. Cuando aparezca la línea `Enemy Team` dejamos de nombrar personajes del equipo del jugador y se empiezan a nombrar las bestias del equipo enemigo, que solo contienen su nombre.

Por ejemplo, un posible equipo:

```
Player Team
Primrose (Lion Dance,Moonlight Waltz,Peacock Strut) [The Show Goes On]
H'aanit (Rain of Arrows,True Strike,Thunderbird,Leghold Trap)
Cyrus [Elemental Augmentation]
Olberic
Enemy Team
Meep
Devourer of Dreams
```

En este caso el jugador tendría 4 personajes en su equipo:

- **Primrose**: Posee 3 habilidades activas, y una habilidad de apoyo.
- **H'aanit**: Posee 4 habilidades activas y ninguna habilidad de apoyo.
- **Cyrus**: No posee ninguna habilidad activa, pero posee una habilidad de apoyo.
- **Olberic**: No posee ningún tipo de habilidad

En cuanto al equipo del enemigo vemos que se conforma por dos bestias: **Meep** y **Devourer of Dreams**.

Formato Unidades

La información de las unidades que participarán en los combates se puede encontrar en los archivos json.

En `characters.json` se entrega la información de los viajeros que el jugador puede utilizar en un combate, junto con sus stats y armas que pueden utilizar para sus ataques básicos. Se utiliza el siguiente formato:

```
[
  {...},
  {
    "Name": "Tressa",
    "Stats": {
      "HP": 3080,
      "SP": 357,
      "PhysAtk": 384,
      "PhysDef": 333,
      "ElemAtk": 360,
      "ElemDef": 285,
      "Speed": 240
    },
    "Weapons": ["Spear", "Bow"]
  },
  {...}
]
```

Por otro lado en el archivo `enemies.json` podrán consultar la información sobre las bestias que formarán el equipo enemigo a los cuales el jugador se puede enfrentar durante el combate. En él se tiene los stats de los enemigos, el nombre de su habilidad, sus debilidades y cantidad de `Shields`.

```
[
  {...},
  {
    "Name": "Meep",
    "Stats": {
      "HP": 1308,
      "PhysAtk": 321,
      "PhysDef": 131,
      "ElemAtk": 327,
      "ElemDef": 77,
      "Speed": 63
    },
    "Skill": "Attack",
    "Shields": 2,
    "Weaknesses": ["Bow","Stave","Dark"]
  },
  {...}
]
```

Formato Habilidades

Habilidades Activas

La información de las habilidades activas se puede encontrar en el archivo `skills.json`. Para cada habilidad se indica su nombre, costo de **SP**, tipo, efecto, target y modificador. Para esta entrega solo se usará el nombre para verificar si un equipo es válido. Se utiliza el siguiente formato:

```
[
  {...},
  {
    "Name": "Fireball",
    "SP": 8,
    "Type": "Fire",
    "Description": "Inflige daño de tipo Fire a todos los enemigos.",
    "Target": "Enemies",
    "Modifier": 1.7,
```

```
    "Boost": "Aumenta el modificador en un 90% por cada BP"
  },
  {
    "Name": "Lion Dance",
    "SP": 4,
    "Type": "",
    "Description": "Incrementa el daño físico de un aliado durante 2 turnos.",
    "Target": "Ally",
    "Modifier": 0,
    "Boost": "Aumenta la duración en 2 turnos por cada BP"
  },
  {...}
]
```

Notemos que el campo *Type* indica el tipo para los ataques (**Fire**, **Ice**, **Axe**, etc), pero en habilidades que no realizan daño (como el caso de `Lion Dance`) este campo queda vacío, ya que no tienen un tipo de ataque asociado.

Habilidades pasivas

La información de las habilidades pasivas se puede encontrar en el archivo `passive_skills.json`. Para cada habilidad pasiva se indica su nombre, `target` y descripción. Al igual que las habilidades activas, en esta entrega solo se utilizará el nombre para verificar si un equipo es válido. Se utiliza el siguiente formato:

```
[
    {...},
    {
      "Name": "Elemental Augmentation",
      "Description": "Aumenta en 50 el stat PhysAtk de la unidad que porte la habilidad",
      "Target": "User"
    },
    {...}
]
```

Habilidades de Bestias

La información sobre las habilidades de bestias se puede encontrar en el archivo `beast_skills.json`, en donde se muestran con nombre, modificador, descripción, objetivo sobre el cual se aplica y cantidad de golpes que realiza.

```
[
  {...},
  {
    "name": "Attack",
    "modifier": 1.3,
    "description": "Realiza un ataque físico al viajero con mayor HP",
    "target": "Single",
    "hits": 1
  },
  {...}
]
```

Habilidades a implementar

Para esta entrega deberás implementar una única habilidad, que corresponde a la habilidad de una bestia:

- `[Phys,1.3,Single,1]`, **Attack**: Realiza un ataque físico al viajero con mayor **HP**.

Para esta entrega pueden asumir que todas las bestias tienen la habilidad `Attack`, la cual tiene un efecto idéntico al ataque básico, pero aplicado sobre el viajero con mayor **HP**.

Output del juego

El programa siempre comienza mostrando los equipos que se pueden elegir dentro de la carpeta `teamsFolder`. Luego de ello, si el equipo es inválido, se notifica al usuario y termina el programa:

```
Elige un archivo para cargar los equipos
0: 000.txt
1: 001.txt
2: 002.txt
3: 003.txt
4: 004.txt
5: 005.txt
INPUT: 0
Archivo de equipos no válido
```

En la ejecución de tests "`INPUT:`" no corresponde a un texto que se deba imprimir, si no que se utiliza en los tests para representar que en esa parte el programa espera que el usuario ingresa un valor.

En el caso contrario, donde el archivo de equipos es válido, se dará inicio a la simulación entre el jugador y el enemigo.

Para los ejemplos de output a continuación supondremos que ejecutamos el programa con el siguiente archivo de equipos, el cual es válido.

```
Player Team
H'aanit (Rain of Arrows, True Strike)
Tressa
Enemy Team
Meep
```

El juego iniciará en la ronda 1, cada vez que se inicie una ronda se deberá mostrar el mensaje de inicio de ronda.

```
----------------------------------------
INICIA RONDA 1
----------------------------------------
```

Luego, se continua con el flujo de la ronda. Antes del turno de cada unidad se deberá mostrar el estado general del juego: primero mostrando el equipo del jugador con sus viajeros y su posición dentro del tablero (utilizando las letras A hasta la D), y luego mostrando al equipo del enemigo con sus respectivas posiciones dentro del tablero (utilizando las letras A hasta la E). Tanto los viajeros como las bestias se sitúan en las casillas según el orden en el que estaban dentro del archivo seleccionado. Si hay posiciones del tablero que no se ocupan estas no se deben mostrar.

Para mostrar el estado del juego se deberán mostrar las unidades con el siguiente formato:

- Viajeros: (Nombre) - HP:(HP actual)/(HP máximo) SP: (SP actual)/(SP máximo) BP:(BP actual)
- Bestias: (Nombre) - HP:(HP actual)/(HP máximo) Shields:(Shields actuales)

Siguiendo el ejemplo tendríamos el siguiente mensaje al iniciar el combate

```
----------------------------------------
INICIA RONDA 1
----------------------------------------
Equipo del jugador
```

```
A-H'aanit - HP:3096/3096 SP:369/369 BP:1
B-Tressa - HP:3080/3080 SP:357/357 BP:1
Equipo del enemigo
A-Meep - HP:1308/1308 Shields:2
```

En el estado del juego siempre se mostrarán todas las unidades, incluso si estas no se encuentran vivas. En este caso la unidad simplemente se mostrara con `HP = 0`. Supongamos el caso en que `Tressa` muere durante el combate, entonces en el siguiente resumen del combate se mostraría

```
----------------------------------------
Equipo del jugador
A-H'aanit - HP:3096/3096 SP:369/369 BP:1
B-Tressa - HP:0/3080 SP:357/357 BP:1
Equipo del enemigo
A-Meep - HP:1308/1308 Shields:2
```

Luego de mostrar el estado, se deberá mostrar el orden de turnos para la ronda actual y la próxima ronda, siguiendo las reglas de ordenamiento para la cola de turnos. El formato para ello será el siguiente

```
----------------------------------------
INICIA RONDA 1
----------------------------------------
Equipo del jugador
A-H'aanit - HP:3096/3096 SP:369/369 BP:1
B-Tressa - HP:3080/3080 SP:357/357 BP:1
Equipo del enemigo
A-Meep - HP:1308/1308 Shields:2
----------------------------------------
Turnos de la ronda
1.H'aanit
2.Tressa
3.Meep
----------------------------------------
Turnos de la siguiente ronda
1.H'aanit
2.Tressa
3.Meep
```

En caso de que una unidad ya haya jugado su turno durante la ronda, está no se mostrará en el orden de turnos de la ronda actual, de modo que la ronda actual solo muestra a las unidades que tienen su turno pendiente. Por ejemplo si `H'aanit` ya ha jugado su turno, entonces podríamos ver el orden de la siguiente manera

```
----------------------------------------
Turnos de la ronda
1.Tressa
2.Meep
----------------------------------------
Turnos de la siguiente ronda
1.H'aanit
2.Tressa
3.Meep
```

Turno Viajero

Luego de mostrar el orden de rondas, iniciará el turno de la unidad correspondiente, en caso de ser el turno de un viajero se le deberá presentar la opción de escoger entre las acciones posibles y solicitar un input para aplicar la acción. El siguiente ejemplo muestra las opciones que se presentan cuando a la viajera `H'aanit` le toca jugar su turno:

```
---------------------------------------
INICIA RONDA 1
---------------------------------------
Equipo del jugador
A-H'aanit - HP:3096/3096 SP:369/369 BP:1
B-Tressa - HP:3080/3080 SP:357/357 BP:1
Equipo del enemigo
A-Meep - HP:1308/1308 Shields:2
---------------------------------------
Turnos de la ronda
1.H'aanit
2.Tressa
3.Meep
---------------------------------------
Turnos de la siguiente ronda
1.H'aanit
2.Tressa
3.Meep
---------------------------------------
Turno de H'aanit
1: Ataque básico
2: Usar habilidad
3: Defender
4: Huir
INPUT:
```

Ataque básico

Una de las acciones que se debe implementar en esta entrega es el ataque básico. Como bien se indica en el enunciado general, esta acción corresponde a un ataque que el viajero realiza con una de sus armas contra uno de los enemigos. Si el jugador opta por realizar un ataque básico, se deberá dar a escoger entre las armas del viajero junto con la opción de cancelar. Las armas se muestran en el orden que las presenta el viajero en el archivo json. En este ejemplo vemos como se presentan las opciones cuando `H''aanit` decide realizar un ataque básico.

```
---------------------------------------
Seleccione un arma
1: Axe
2: Bow
3: Cancelar
INPUT:
```

Si se escoge un arma, entonces se le dará a escoger entre las bestias vivas para seleccionar el objetivo de su ataque. Al momento de presentar las bestias estas mostrarán sus datos de la misma forma y orden que en el estado del juego.

```
---------------------------------------
Seleccione un objetivo para H'aanit
1: Meep - HP:1308/1308 Shields:2
2: Cancelar
INPUT:
```

Finalmente, se preguntará si se desea usar **BP** para realizar el ataque con boosting.

```
---------------------------------------
Seleccione cuantos BP utilizar
INPUT:
```

Esta última sección solo se mostrará en caso de que el viajero que realiza el ataque tiene por lo menos 1 **BP**, en caso de tener 0 **BP** no se preguntará, ya que no puede realizar boosting. Para esta entrega se podrá

asumir que cada vez que se pregunte por **BP**, los tests ingresaran como input el valor 0, de modo que no es necesario implementar la mecánica de boost.

Dado las selecciones del jugador, se realiza el ataque y se muestra el resumen de la acción, mencionando la unidad que efectúa el ataque, la unidad que lo recibe, el daño realizado, el arma utilizada y la vida restante de la unidad atacada. Un ejemplo de este sección:

```
---------------------------------------
H'aanit ataca
Meep recibe 373 de daño de tipo Axe
Meep termina con HP:194
```

Luego de este mensaje, se pasa al siguiente turno, mostrando nuevamente el estado del juego, el orden de las colas de turno y mostrando el turno de la siguiente unidad para jugar. Se muestra un ejemplo de la secuencia completa en donde `H'aanit` realiza un ataque básico usando **Axe** sobre `Meep`, sin utilizar boosting:

```
---------------------------------------
Turno de H'aanit
1: Ataque básico
2: Usar habilidad
3: Defender
4: Huir
INPUT: 1
---------------------------------------
Seleccione un arma
1: Axe
2: Bow
3: Cancelar
INPUT: 1
---------------------------------------
Seleccione un objetivo para H'aanit
1: Meep
2: Cancelar
INPUT: 1
---------------------------------------
Seleccione cuantos BP utilizar
INPUT: 0
---------------------------------------
H'aanit ataca
Meep recibe 373 de daño de tipo Axe
Meep termina con HP:935
```

En caso de que el cálculo de daño de como resultado 0 el programa deberá mostrar que el ataque se realizo de igual forma, indicando que se inflige 0 de daño

```
---------------------------------------
H'aanit ataca
Meep recibe 0 de daño de tipo Axe
Meep termina con HP:1308
```

Usar Habilidad

Si el jugador escoge usar una habilidad entonces se deberá dar a elegir entre las habilidades que es posible utilizar en el momento. Habilidades que no se puedan utilizar por que la unidad no tiene suficiente `SP` no deben ser mostradas. Un ejemplo de selección de habilidad para `H'aanit` se vería de la siguiente manera

```
---------------------------------------
Seleccione una habilidad para H'aanit
1: Rain of Arrows
2: True Strike
3: Cancelar
```

```
INPUT:
```

Para esta entrega no se les pedirá implementar habilidades, por lo que puedes asumir que llegado a este punto siempre se escogerá la opción `Cancelar`. También podrás asumir que todas las unidades poseen `SP` suficiente para usar todas sus habilidades activas, por tanto para esta entrega siempre se mostrarán todas las habilidades activas.

Cancelar

En cualquier paso del flujo, si el jugador escoge `Cancelar` se deberá mostrar el menú de selección de acción. Por ejemplo, si presiona cancelar a la hora de escoger un objetivo se tendría el siguiente output.

```
----------------------------------------
Turno de H'aanit
1: Ataque básico
2: Usar habilidad
3: Defender
4: Huir
INPUT: 1
----------------------------------------
Seleccione un arma
1: Axe
2: Bow
3: Cancelar
INPUT: 1
----------------------------------------
Seleccione un objetivo para H'aanit
1: Meep
2: Cancelar
INPUT: 2
----------------------------------------
Turno de H'aanit
1: Ataque básico
2: Usar habilidad
3: Defender
4: Huir
INPUT:
```

Huir

La siguiente acción que se deberá implementar en esta entrega corresponde a **Huir**, la cual simplemente acaba el combate dando por ganador al equipo enemigo, en caso de que el jugador opte por huir se debera mostrar:

```
----------------------------------------
Turno de H'aanit
1: Ataque básico
2: Usar habilidad
3: Defender
4: Huir
INPUT: 4
----------------------------------------
El equipo de viajeros ha huido!
```

Luego de esto finaliza el combate y se muestra el mensaje de ganador.

Turno Bestia

Cuando se ejecuta el turno de la bestia, no se le debe preguntar nada al usuario, ya que el enemigo es controlado por la computadora. El programa deberá anunciar que la bestia utilizará su habilidad y posteriormente

mostrar los efectos de esta. Un ejemplo de `Meep` que posee la habilidad `Attack` jugando su turno se vería de la siguiente manera

```
----------------------------------------
Meep usa Attack
Tressa recibe 84 de daño físico
Tressa termina con HP:2996
```

Ganador

Cuando se acabe el combate se debe notificar el ganador. Existen 3 casos en donde finaliza el combate:

- Todas las bestias mueren, dando por ganador al jugador.
- Todos los viajeros mueren, dando por ganador al enemigo.
- Jugador huye del combate, dando por ganador al enemigo.

Si el ganador resulta ser el jugador se deberá notificar:

```
----------------------------------------
Gana equipo del jugador
```

En caso de que sea el enemigo quien gana, se deberá notificar:

```
----------------------------------------
Gana equipo del enemigo
```

Un ejemplo en donde el jugador gana, dado que los enemigos mueren pordría ser el siguiente:

```
----------------------------------------
H'aanit ataca
Meep recibe 373 de daño de tipo Axe
Meep termina con HP:0
----------------------------------------
Gana equipo del jugador
```

Un ejemplo en donde el jugador huye y pierde el combate podría ser el siguiente:

```
----------------------------------------
Seleccione una acción para Tressa
1: Ataque básico
2: Usar habilidad
3: Defender
4: Huir
INPUT: 4
----------------------------------------
El equipo de viajeros ha huido!
----------------------------------------
Gana equipo del enemigo
```

Cálculo de daño

Tal como indica el enunciado general del proyecto, los cálculos de daño pueden generar números decimales. Cuando ello ocurre, hay que truncar el número a su entero más bajo. Esto se puede realizar en C# utilizando la función `Math.Floor(...)`. Luego el resultado puede ser convertido a entero con `Convert.ToInt32(...)`.

Input-Output

En tu proyecto **NO** debes usar `Console.WriteLine(...)` ni `Console.ReadLine()` para mostrar y pedir texto al usuario. Esto se debe a que nuestro código para comparar la salida de tu programa con los test cases ignora los mensajes mandados directamente a consola.

Para que el input-output de tu programa sea verificado por nuestros test cases debes usar el objeto `view` que te entregamos en el constructor de `Game.cs`. Ese objeto tiene los siguientes métodos:

- `ReadLine()`: Solicita un string al usuario y retorna el `string` correspondiente.
- `WriteLine(string message)`: Muestra `message` en consola.

El objeto `view` hace dos cosas. Por un lado, guarda los mensajes que se escriben mediante su método `WriteLine(...)`. Esos mensajes son comparados con el test case para verificar si tu programa está correcto. Por otro lado, cuando se llama a `ReadLine()` automáticamente retorna el `INPUT:` indicado en el test case.

En resumen, todo input pedido y mensaje mostrado mediante `Console` es ignorado al momento de evaluar los test cases. Si quieres que un input o texto sea considerado debes utilizar los métodos de `view`.

Rúbrica

Para evaluar tu entrega usaremos 3 grupos de test cases: `E1-RandomBasicCombat`, `E1-InvalidTeams` y `E1-BasicCombat`.

Esta entrega tiene puntaje por funcionalidad y por limpieza de código. Para calcular tu puntaje de funcionalidad se le asignará a cada grupo de test un puntaje máximo, el cual será el limite superior del puntaje que obtendrás en dicho grupo de tests; el puntaje que obtengas variará proporcionalmente a la cantidad de tests del grupo que pases. Los puntajes se distribuyen de la siguiente manera:

- **[0.7 puntos]** Porcentaje de test cases pasados en `E1-InvalidTeams`.
- **[3.0 puntos]** Porcentaje de test cases pasados en `E1-BasicCombat`.
- **[2.3 puntos]** Pasar todos los test cases en `E1-RandomBasicCombat`.

Por ejemplo, digamos que tu entrega todos los test cases `E1-InvalidTeams` , todos los test cases `E1-RandomBasicCombat` y el 80 % de los test cases `E1-BasicCombat`. Entonces tu puntaje de funcionalidad será: $0{,}7+2{,}3+3{,}0\cdot 0{,}8 = 5{,}4$.

Por otro lado, el puntaje por limpieza de código es en base a descuentos. Es decir, se parte con 6 puntos y se descuenta en base a las violaciones de los principios de los capítulos de Clean Code que presente tu código. Los descuentos máximos por capítulo son los siguientes:

- **[-2.0 puntos] No** sigue los principios del cap. 2 de *clean code*.
- **[-2.5 puntos] No** sigue los principios del cap. 3 de *clean code*.

Finalmente, tu nota final será igual al promedio geométrico entre el puntaje por funcionalidad y el puntaje por limpieza de código (más el punto base), donde el promedio geométrico entre x e y es igual a $\sqrt{xy}$.

Por ejemplo, si tienes 3 puntos por funcionalidad y 5 puntos por limpieza de código entonces tu nota será $\sqrt{3\cdot 5}+1 = 4{,}9$. Pero si tienes 6 puntos en funcionalidad y 1.5 en limpieza de código entonces tu nota será $\sqrt{6\cdot 1{,}5}+1 = 4{,}0$.

Importante: No está permitido modificar los test cases ni el proyecto `Octopath-Traveler.Tests`. Hacerlo puede conllevar una penalización que dependerá de la gravedad de la situación